

DESIGN BY CONTRACT

O artigo “Design by Contract”, escrito por Bertrand Meyer, apresenta uma das propostas mais influentes da engenharia de software orientada a objetos. O autor parte da ideia de que sistemas confiáveis não podem depender apenas da habilidade individual dos programadores, mas sim de mecanismos formais capazes de definir claramente as responsabilidades de cada componente. A metáfora central do texto compara o desenvolvimento de software à criação de contratos jurídicos, nos quais existem obrigações, garantias e limites bem estabelecidos para cada parte envolvida. Essa visão transforma rotinas e classes em entidades que possuem deveres explícitos diante de seus clientes.

Meyer inicia o texto discutindo o problema da confiabilidade de software. Ele argumenta que a indústria frequentemente prioriza flexibilidade e modularidade sem dedicar a mesma atenção à corretude. Segundo o autor, isso se torna ainda mais grave em sistemas orientados a objetos, pois a reutilização exige um nível elevado de confiança nos componentes compartilhados entre aplicações diferentes. Um componente reutilizável defeituoso pode espalhar problemas para dezenas de sistemas distintos. Por isso, a confiabilidade passa a ser requisito estrutural da orientação a objetos.

A primeira grande contribuição do artigo é a definição do conceito de contrato aplicado ao software. Quando um módulo utiliza outro módulo, existe uma relação semelhante à de cliente e fornecedor. O cliente possui certas obrigações antes de realizar a chamada de uma rotina, enquanto o fornecedor assume o compromisso de entregar um resultado específico após a execução. Meyer mostra que a ausência de contratos explícitos gera ambiguidades, interpretações diferentes e arquiteturas frágeis.

No modelo apresentado, as pré-condições definem o que deve ser verdadeiro antes da execução de uma rotina. Já as pós-condições definem aquilo que a rotina promete garantir ao final do processamento. Além disso, invariantes de classe representam propriedades permanentes que precisam permanecer válidas nos estados observáveis do objeto. Essa divisão cria um sistema disciplinado de responsabilidades, permitindo identificar de forma objetiva quem violou o contrato em caso de falha.

Os exemplos em Eiffel ajudam bastante na compreensão da teoria. Em um dos casos apresentados, uma tabela possui uma rotina de inserção que só aceita chamadas quando ainda há espaço disponível. Em troca, garante que o elemento foi corretamente armazenado e que o número de itens aumentou. O interessante é perceber que Meyer trata essas cláusulas como partes oficiais do software, e não apenas comentários informais escritos pelos desenvolvedores. Um dos pontos mais importantes do artigo é a crítica à programação defensiva tradicional. Durante décadas, muitos programadores acreditaram que cada função deveria validar absolutamente todas as possibilidades de erro. Meyer considera essa abordagem prejudicial porque aumenta excessivamente a complexidade do sistema. Segundo ele, quando cliente e fornecedor verificam as mesmas condições, ocorre redundância desnecessária. Isso produz códigos extensos, difíceis de manter e repletos de verificações repetitivas.

A alternativa defendida pelo autor é muito mais organizada. Cada condição deve possuir um responsável específico. Se determinada regra faz parte da pré-condição, então o cliente

precisa garanti-la. Caso contrário, o fornecedor assume a obrigação de lidar com a situação. Essa divisão elimina verificações redundantes e simplifica drasticamente a arquitetura do sistema.

Essa crítica continua extremamente atual. Muitos projetos modernos ainda sofrem com excesso de validações espalhadas em diferentes camadas da aplicação. É comum encontrar APIs, serviços e interfaces repetindo exatamente as mesmas verificações, criando inconsistências e dificultando manutenção. O Design by Contract propõe justamente o contrário: clareza arquitetural baseada em contratos bem definidos. Outro tema importante do artigo é a definição de invariantes de classe. Meyer explica que um objeto não deve apenas funcionar corretamente durante uma rotina específica, mas também manter consistência ao longo de toda sua existência. O invariante representa uma propriedade permanente da classe. Em uma estrutura de tabela, por exemplo, o número de elementos nunca pode ultrapassar a capacidade máxima disponível.

O autor utiliza um diagrama do ciclo de vida do objeto para explicar essa ideia. Após a criação do objeto e após a execução de qualquer rotina exportada, o invariante precisa continuar verdadeiro. Entre esses estados observáveis, entretanto, o sistema pode atravessar estados temporariamente inconsistentes durante o processamento interno. Essa observação é importante porque evita interpretações excessivamente rígidas do conceito de consistência. A relação entre invariantes e contratos jurídicos também é bastante interessante. Meyer compara invariantes a cláusulas gerais presentes em contratos do mundo real. Elas se aplicam continuamente a todas as operações realizadas dentro daquele contexto. Dessa forma, toda rotina exportada precisa respeitar essas regras permanentes da classe.

O artigo também apresenta uma definição formal de corretude de classes. Meyer utiliza conceitos inspirados na verificação formal para explicar que uma classe é considerada correta quando todas as suas rotinas preservam os invariantes e cumprem suas pré e pós-condições.

O texto introduz fórmulas lógicas que relacionam estados iniciais, execução e estados finais, aproximando o Design by Contract de métodos matemáticos de prova de programas. Apesar do formalismo presente em alguns trechos, Meyer consegue manter uma abordagem relativamente acessível. Ele evita transformar o artigo em um texto puramente matemático e sempre procura conectar os conceitos formais com problemas concretos de desenvolvimento de software. Isso torna a leitura relevante tanto para pesquisadores quanto para profissionais de mercado. Outro mérito do artigo é mostrar que contratos funcionam também como mecanismo de documentação. O comando `short` do ambiente Eiffel extrai automaticamente as informações públicas de uma classe, incluindo pré-condições e pós-condições. Assim, a documentação deixa de ser um artefato separado e passa a ser derivada diretamente do código-fonte.

A discussão sobre assertions em tempo de execução é outro ponto extremamente relevante. Meyer explica que o ambiente Eiffel permite diferentes níveis de monitoramento, variando desde nenhuma verificação até checagem completa de pré-condições, pós-condições e invariantes. Quando uma assertion falha, o sistema dispara uma exceção indicando que alguma hipótese assumida pelos desenvolvedores foi violada.

O autor insiste que uma violação de assertion não representa um caso normal de execução, mas sim um bug de especificação, implementação ou uso incorreto do componente. Essa visão é importante porque diferencia erros genuínos de condições normais de negócio. Muitas aplicações modernas confundem essas categorias, utilizando exceções para representar eventos previsíveis e comuns.

Meyer também apresenta a ideia do “paradoxo da semântica das assertions”. Em teoria, sistemas corretos nunca deveriam violar assertions. Entretanto, na prática, o monitoramento dessas assertions é justamente um dos melhores mecanismos para descobrir se o sistema realmente está correto. Essa observação mostra como Design by Contract combina teoria formal com pragmatismo de engenharia.

A parte do artigo dedicada ao tratamento de exceções é provavelmente uma das mais críticas e polêmicas do texto. Meyer critica fortemente mecanismos tradicionais encontrados em linguagens como Ada e CLU. Segundo ele, muitas abordagens utilizavam exceções de maneira excessiva e desorganizada, transformando-as em simples estruturas de controle de fluxo.

O autor apresenta exemplos concretos de códigos considerados problemáticos. Em um deles, uma rotina de raiz quadrada captura uma exceção causada por número negativo, imprime uma mensagem de erro e continua a execução normalmente. Meyer considera esse comportamento extremamente perigoso, pois o cliente da rotina não é informado adequadamente sobre a falha. O sistema continua funcionando com resultados inválidos.

A crítica central é que uma rotina não pode fingir que cumpriu seu contrato quando na verdade falhou. Isso leva Meyer a formular leis formais para o tratamento de exceções. A primeira afirma que existem apenas duas formas legítimas de terminar uma chamada: ou a rotina cumpre seu contrato, ou falha explicitamente. Não existe espaço para retornos silenciosos mascarando erros.

O conceito de “organized panic” apresentado por Meyer é bastante interessante. Quando uma rotina não consegue cumprir seu contrato, ela deve pelo menos restaurar o sistema para um estado consistente antes de sinalizar a falha ao cliente. Em Eiffel, isso é realizado por meio das cláusulas rescue e retry.

A cláusula rescue define um comportamento alternativo caso uma exceção ocorra durante a execução principal. Se o código de recuperação conseguir resolver o problema, a instrução retry reinicia a rotina. Caso contrário, a rotina falha oficialmente e propaga a exceção para o chamador. Esse modelo cria um mecanismo disciplinado de recuperação de falhas.

Os exemplos fornecidos pelo artigo demonstram bem a proposta. Um deles envolve leitura de inteiros fornecidos por um usuário. Caso a entrada seja inválida, o sistema solicita novamente um valor correto até um limite de tentativas. Outro exemplo mostra uma rotina que tenta calcular o inverso de um número real, retornando zero caso a divisão gere erro numérico. Outro aspecto extremamente importante do texto é a relação entre contratos e herança. Meyer argumenta que redefinições de métodos em subclasses precisam respeitar regras específicas para preservar a segurança do polimorfismo. O autor interpreta herança

como uma forma de subcontratação. Uma subclasse atua como um subcontratado que assume as responsabilidades originalmente definidas pela superclasse. Nesse contexto, o subcontratado não pode exigir mais do cliente do que o contrato original exigia. Portanto, pré-condições em subclasses devem ser iguais ou mais fracas que as pré-condições originais. Ao mesmo tempo, o subcontratado também não pode entregar menos do que foi prometido inicialmente. Assim, pós-condições precisam ser iguais ou mais fortes.

Essa formulação resolve diversos problemas clássicos da orientação a objetos. Sem essas restrições, subclasses poderiam quebrar expectativas assumidas por clientes genéricos, comprometendo completamente a segurança do polimorfismo. A contribuição conceitual de Meyer nesse ponto foi extremamente importante para a evolução da teoria orientada a objetos.

A discussão sobre dynamic binding também é muito relevante. Meyer considera o binding dinâmico essencial para o funcionamento correto da orientação a objetos. Quando um cliente utiliza uma referência genérica, o sistema deve escolher automaticamente a implementação apropriada conforme o tipo real do objeto em tempo de execução.

O autor critica fortemente o static binding em situações nas quais existem redefinições válidas em subclasses. Segundo ele, aplicar a versão errada de uma rotina pode quebrar invariantes e produzir estados inconsistentes. A defesa do dynamic binding está diretamente ligada ao respeito aos contratos estabelecidos pelas classes.

Essa análise demonstra como o artigo vai muito além de simples validações de entrada e saída. Meyer constrói uma teoria completa envolvendo corretude, modularidade, herança, exceções e confiabilidade arquitetural. O conceito de contrato funciona como elemento integrador de toda a proposta. Outro mérito do artigo é sua enorme influência histórica. Muitas ideias modernas relacionadas a APIs, interfaces, validações automáticas e especificações formais possuem relação direta com Design by Contract. Mesmo linguagens que nunca implementaram contratos formalmente acabaram absorvendo vários princípios apresentados por Meyer.

Frameworks modernos frequentemente utilizam mecanismos equivalentes a pré-condições e pós-condições. Sistemas de tipagem forte, validações automáticas e testes unitários também refletem parte da filosofia defendida pelo autor. Além disso, práticas modernas de desenvolvimento orientado a domínio valorizam fortemente contratos explícitos entre componentes.

O impacto acadêmico também foi enorme. O artigo influenciou discussões sobre semântica de linguagens, verificação formal, herança segura e tratamento disciplinado de exceções. Diversos pesquisadores passaram a explorar maneiras de incorporar contratos diretamente nas linguagens de programação.

Apesar de suas qualidades, o texto também possui limitações. Em alguns momentos, Meyer parece assumir um ambiente relativamente controlado, típico de sistemas orientados a objetos tradicionais. Em arquiteturas modernas distribuídas, baseadas em microsserviços e concorrência massiva, contratos podem ser mais difíceis de garantir devido a falhas externas, latência de rede e comunicação assíncrona. Outro possível problema é o aumento

de formalidade exigido pelo modelo. Em projetos pequenos ou protótipos rápidos, muitos desenvolvedores podem considerar excessivo o esforço necessário para especificar contratos detalhados. Além disso, equipes sem forte cultura de engenharia podem acabar ignorando ou utilizando incorretamente essas especificações. Mesmo assim, a essência do artigo continua extremamente válida. A ideia de explicitar responsabilidades entre componentes permanece fundamental em qualquer sistema complexo. Muitas falhas arquiteturais modernas surgem justamente da ausência de contratos claros.

Em síntese, “Design by Contract” é uma das obras mais importantes da engenharia de software orientada a objetos. Bertrand Meyer consegue unir teoria formal e prática de programação em uma proposta extremamente coerente. O artigo apresenta contratos como mecanismo central para obtenção de software confiável, reutilizável e arquiteturalmente consistente.

Os conceitos de pré-condições, pós-condições e invariantes fornecem uma estrutura clara para distribuição de responsabilidades entre clientes e fornecedores de componentes. A crítica à programação defensiva excessiva continua extremamente atual. Da mesma forma, a abordagem disciplinada para tratamento de exceções oferece uma alternativa elegante aos mecanismos caóticos encontrados em muitas linguagens. Além disso, a interpretação da herança como subcontratação representa uma das explicações mais sofisticadas já produzidas para o polimorfismo orientado a objetos. O artigo demonstra que reuso seguro depende de contratos preservados ao longo das hierarquias de classes.

Por fim, o trabalho de Meyer mostra que confiabilidade não surge do acaso nem de verificações improvisadas espalhadas pelo sistema. Ela depende de regras claras, responsabilidades explícitas e contratos bem definidos. Mesmo décadas após sua publicação, o texto continua sendo leitura fundamental para qualquer profissional interessado em arquitetura de software, orientação a objetos e construção de sistemas robustos.